```
>>> match = re.match(r "my (.*) is (.*)", "my name is
david");
>>> match.groups()
('name', 'david')
```

We get a successful match in this case, since the start of the string matches the regex.

re.findall() & re.finditer()

Of all things regex, `findall()` is probably the most practical and useful function provided with this module. As the name suggests, it finds all matches of the regex in the provided string which are non-overlapping and returns them as a list of strings, each string representing a match.

```
>>> str= "abc@abc.com, abc#xyz.com, sarah@dummy.net,
david@goliath.co"
>>> re.findall(r'[\w\.-]+@[\w\.-]+', str)
['abc@abc.com', 'sarah@dummy.net', 'david@goliath.co']
```

`findall()` method finds all email addresses in the string str. The regex supplied looks for any string with an '`@`' surrounded by a word, '`.`' `or a` '`-`' on either side.

The `finditer()` function in this module is quite similar to the `findall()`, except the fact that it returns an iterator of match objects, instead of a list of matched strings. We can iterate through the match objects and get all the match groups. It should be noted that iterators can only be iterated through once, post that match objects will not be available for iteration.

```
>>> finditer = re.finditer(r'[\w\.-]+@[\w\.-]+', str)
>>> for match in finditer:
...     match.group()
...
'abc@abc.com'
'sarah@dummy.net'
'david@goliath.co'
```

asyncio

`asyncio` was added as a provisional module in Python 3.4 and has been a part since then. It provisions single-threaded concurrent code execution using coroutines, event loops, futures and tasks. It is necessary that you